*** DAY 3 Class Work***

Completed by: Anthony Bayate - Web Client-Side Programming, Kean University

Complete this Python refresher, fill in the blanks, fix the errors if appropriate

Trace cell by cell do not just run all

In [1]:
```
print("Functions allow us to reuse code and make our programs more modular.")
```

```
Functions allow us to reuse code and make our programs more modular.
```

Slide 2-3: Defining and Calling a Simple Function

In [2]:
```
def greet():
    print("Hello! Welcome to Python Functions.")

# Run the function
greet()
```

```
Hello! Welcome to Python Functions.
```

Challenge 1

In [3]:
```
# Challenge 1: Define a function that prints "Python is fun!"
def python_fun():
    print("Python is fun!")

python_fun()
```

```
Python is fun!
```

Slides 6-7: Function with Parameters

In [4]:
```
def personalized_greeting(name):
    print(f"Hello, {name}! Welcome to Python Functions.")

# Run the function
personalized_greeting("Anthony")
```

```
Hello, Anthony! Welcome to Python Functions.
```

Challenge 2: Define a function that takes two arguments and prints their sum

In [5]:
```
def print_sum(a, b):
    print(f"{a} + {b} = {a + b}")

print_sum(7, 5)
```

```
7 + 5 = 12
```

Slide 7: Returning Values from Functions

In [6]:
```python
def add(a, b):
    return a + b

# Run the function and print the result
result = add(3, 5)
print(f"The sum is: {result}")
```

```
The sum is: 8
```

Challenge 3: Define a function that returns the product of two numbers

In [7]:
```python
def multiply(a, b):
    return a * b

print(f"6 x 7 = {multiply(6, 7)}")
```

```
6 x 7 = 42
```

Slide 8-9: Local and Global Variables

In [8]:
```python
# Example 4: Local Variable
def local_variable_example():
    local_var = 10
    print(f"Local variable: {local_var}")

# Run the function
local_variable_example()
```

```
Local variable: 10
```

Challenge 4: Define a function that declares a local variable and prints it

In [9]:
```python
def print_local():
    my_name = "Anthony Bayate"   # local: only exists inside this function
    print(f"Local variable: {my_name}")

print_local()
```

```
Local variable: Anthony Bayate
```

Slide 10: Global Variable

In [10]:
```python
global_var = 20

def global_variable_example():
    print(f"Global variable: {global_var}")

# Run the function  (fixed: was "#lobal_variable_example()")
global_variable_example()
```

```
Global variable: 20
```

Challenge 5: Modify the global variable within a function

In [11]:
```python
def modify_global():
    global global_var
    global_var = global_var + 10
    print(f"Global variable inside function: {global_var}")

print(f"Before: {global_var}")
modify_global()
print(f"After: {global_var}")
```

```
Before: 20
Global variable inside function: 30
After: 30
```

Slide 11-13: Passing Arguments to Functions

In [12]:
```python
# Example 6: Passing Multiple Arguments
def print_full_name(first_name, last_name):
    print(f"Full name: {first_name} {last_name}")

# Run the function
print_full_name("Anthony", "Bayate")
```

```
Full name: Anthony Bayate
```

Challenge 6: Define a function that takes three arguments and prints them

In [13]:
```python
def print_three(a, b, c):
    print(f"First: {a}, Second: {b}, Third: {c}")

print_three("Anthony", "Kean University", 2026)
```

```
First: Anthony, Second: Kean University, Third: 2026
```

Slide 14-15: Keyword Arguments

In [14]:
```python
def describe_pet(animal_type, pet_name):
    print(f"\nI have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name}.")

# Run the function using keyword arguments
describe_pet(animal_type='hamster', pet_name='Harry')
```

```
I have a hamster.
My hamster's name is Harry.
```

Challenge 7: Define a function with keyword arguments and call it

In [15]:
```python
def show_vehicle(type, brand):
    print(f"Anthony drives a {brand} {type}.")

# called with keyword arguments, so the order doesn't matter
show_vehicle(brand="Honda", type="sedan")
```

```
Anthony drives a Honda sedan.
```

Slide 10-12: Default Argument Values

In [16]:
```
# Example 8: Default Argument
def print_greeting(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

# Run the function with and without the default argument
print_greeting("Anthony")
print_greeting("Anthony", "Hi")
```

```
Hello, Anthony!
Hi, Anthony!
```

Challenge 8: Define a function with a default argument and call it

In [17]:
```
def introduce(name, age=25):
    print(f"My name is {name} and I am {age} years old.")

introduce("Anthony Bayate")            # uses the default age
introduce("Anthony Bayate", age=21)    # overrides the default
```

```
My name is Anthony Bayate and I am 25 years old.
My name is Anthony Bayate and I am 21 years old.
```

Slide 13: Value-Returning Functions

In [18]:
```
# Example 9: Value-Returning Function
def square(number):
    return number * number

# Run the function and print the result
result = square(4)
print(f"The square of 4 is: {result}")
```

```
The square of 4 is: 16
```

Challenge 9: Define a function that returns the cube of a number

In [19]:
```
def cube(number):
    return number ** 3

print(f"The cube of 3 is: {cube(3)}")
```

```
The cube of 3 is: 27
```

Slides 14-16: The random and math Modules

In [20]:
```
import random
# Generate a random float in the range [0.0, 1.0)
number = random.random()
print("Random float between 0.0 and 1.0:", number)
```

```
Random float between 0.0 and 1.0: 0.13534721991121246
```

In [21]:
```
import random
# Generate a random integer between 1 and 100
```

```
number = random.randint(1, 100)
print("Random number between 1 and 100:", number)
```

```
Random number between 1 and 100: 51
```

In [22]:
```
import math

# Example 10: Using the math Module
def calculate_circle_area(radius):
    return math.pi * radius * radius

# Run the function and print the result
area = calculate_circle_area(5)
print(f"The area of the circle is: {area:.2f}")
```

```
The area of the circle is: 78.54
```

Challenge 10: Define a function that uses the math module to calculate the square root of a number

In [23]:
```
import math

def calculate_sqrt(number):
    return math.sqrt(number)

print(f"The square root of 144 is: {calculate_sqrt(144)}")
```

```
The square root of 144 is: 12.0
```

Slide 20-21: Storing Functions in Modules

In [24]:
```
# Example 11: Creating and Using Modules
# Create a file named my_module.py with the following content:
# def my_function():
#     print("Hello from my_module!")

# from my_module import my_function
# my_function()
#  Using Functions from a Module in Different Files
# Create another file named main.py with the following content:
# from my_module import my_function

# my_function()
```

uncommented the 2 cells below

In [25]:
```
%%writefile my_module.py
def my_function():
    print("Hello from my_module!")
```

```
Writing my_module.py
```

In [26]:
```
from my_module import my_function
my_function()
```

```
Hello from my_module!
```

Challenge 11: Create your own module with at least two functions and import it here

In [27]:
```
%%writefile custom_module.py
# Anthony Bayate's custom module

def function_one(name="Anthony Bayate"):
    print(f"Function One: Hello, {name}!")

def function_two(a, b):
    print(f"Function Two: {a} + {b} = {a + b}")
    return a + b
```

```
Writing custom_module.py
```

In [28]:
```
# Import and use the functions from custom_module
from custom_module import function_one, function_two
function_one()
function_two(10, 32)
```

```
Function One: Hello, Anthony Bayate!
Function Two: 10 + 32 = 42
```

Out[28]: 42

In [29]:
```
# Example 14: Generating Random Numbers
import random

def roll_die():
    return random.randint(1, 6)

# Run the function and print the result
die_roll = roll_die()
print(f"Rolled a die: {die_roll}")
```

```
Rolled a die: 4
```

Challenge 13: Define a function that generates a random float between two numbers

In [30]:
```
import random

def random_float(min_value, max_value):
    return random.uniform(min_value, max_value)

print(f"Random float between 1.5 and 9.5: {random_float(1.5, 9.5):.3f}")
```

```
Random float between 1.5 and 9.5: 7.945
```

In [31]:
```
# Example 15: Returning Multiple Values
def min_max(numbers):
    return min(numbers), max(numbers)

# Run the function and print the result
min_val, max_val = min_max([1, 2, 3, 4, 5])
print(f"Min: {min_val}, Max: {max_val}")
```

```
Min: 1, Max: 5
```

Challenge 14: Define a function that returns the sum, difference, and product of two numbers

In [32]:
```
def calculate(a, b):
    return a + b, a - b, a * b

s, d, p = calculate(12, 4)
print(f"Sum: {s}, Difference: {d}, Product: {p}")
```

```
Sum: 16, Difference: 8, Product: 48
```

Returning None from a Function

In [33]:
```
def divide(num1, num2):
    if num2 == 0:
        return None
    else:
        return num1 / num2

# Run the function and handle the result
result = divide(10, 0)
if result is None:
    print("Cannot divide by zero!")
else:
    print(f"Result: {result}")
```

```
Cannot divide by zero!
```

Challenge 15: Define a function that returns None if a number is negative

In [34]:
```
def check_positive(number):
    if number < 0:
        return None
    return number

for n in (15, -3):
    result = check_positive(n)
    if result is None:
        print(f"{n} is negative -> None")
    else:
        print(f"{n} is positive -> {result}")
```

```
15 is positive -> 15
-3 is negative -> None
```

Conditional Execution of the main Function

In [35]:
```
def main():
    print("This is the main function.")

def secondary_function():
    print("This is a secondary function.")

if __name__ == '__main__':
    main()
```

```
This is the main function.
```

Challenge 16: Define two functions and conditionally execute one based on a variable value

In [36]:
```python
def function_one():
    print("Function one: Anthony is studying.")

def function_two():
    print("Function two: Anthony is taking a break.")

hours_studied = 3
if hours_studied < 2:
    function_one()
else:
    function_two()
```

```
Function two: Anthony is taking a break.
```

In [37]:
```python
%%writefile nasa.py
# apod.py
import requests
from IPython.display import display, Image

def get_apod(api_key):
    response = requests.get(f'https://api.nasa.gov/planetary/apod?api_key=
{api_key}')
    if response.status_code == 200:
        data = response.json()
        return data['title'], data['url'], data['explanation'],
data.get('media_type', 'image')
    else:
        return "Could not fetch APOD data", "", "", ""

def display_apod(api_key):
    title, url, explanation, media_type = get_apod(api_key)
    if title == "Could not fetch APOD data":
        print(title)
    else:
        print(f"Title: {title}\nURL: {url}\nExplanation: {explanation}")
        # fixed: some days the APOD is a video, which Image() cannot show
        if media_type == "image":
            display(Image(url=url, embed=True))
        else:
            print(f"(Today's APOD is a {media_type}, open the URL above to watch
it.)")
```

```
Writing nasa.py
```

check the files created

In [38]:
```python
# Import the apod module
import nasa

# Use the DEMO_KEY provided by NASA
api_key = "DEMO_KEY"
```

```
# Call the function to display APOD data and image
nasa.display_apod(api_key)
```

```
Title: NASA Science
URL: https://science.nasa.gov/wp-content/themes/nasa-child/assets/images/nasa-logo@2x.png
Explanation: This stairway in Chile brings you only a little bit closer to heaven. But ooh, the sky above it makes you wonder. For example, you may wonder about the dark lane of our own Milky Way galaxy rising behind the expansive Atacama Desert in Chile, the band of zodiacal light on the upper left, the green and red airglow just above the horizon, the star Spica in the zodiacal light stripe, and, at the very top on the right, the Small Magellanic Cloud galaxy. The Desert Staircase sculpture is in the foreground, and this image was timed to catch the picturesque peering perch just in front of the near-vertical Milky Way band. Upon close inspection of the horizon you can find, left to right, some Andes Mountains, the Licancabur volcano, and the town lights of San Pedro de Atacama. The featured wide-angle composite was captured in July 2025.Tomorrow's picture: space bees
```



Create your own prompts to test ChatGPT in each of the following sections. Ask instructor for api key if the one proivided here does not work

In [39]:
```
from openai import OpenAI

# Hard-coded API key provided by the instructor for this class activity
client = OpenAI(api_key="sk-proj-...REDACTED (instructor key)")

# Set-up prompt (testing that it works as given)
completion = client.chat.completions.create(
    model="gpt-4.1",
    messages=[
        {
            "role": "user",
            "content": "Write a one-sentence bedtime story about a unicorn."
        }
    ]
)

print(completion.choices[0].message.content)
```

```
Under the silver glow of the moon, a gentle unicorn tiptoed through dreamland meadows, planting magical wishes with every step.
```

In [40]:
```
# My own prompt (retest)
completion = client.chat.completions.create(
    model="gpt-4.1",
    messages=[
        {"role": "system", "content": "You are a friendly study coach for college students."},
        {"role": "user", "content": "My name is Anthony Bayate and I'm a Kean
```

```
University student taking Web Client-Side Programming. "
                                    "Give me 3 short tips for learning JavaScript
frameworks like Vue and Svelte."}
    ]
)

print(completion.choices[0].message.content)
```

Absolutely, Anthony! Here are three quick tips for learning JavaScript frameworks like Vue and Svelte:

1. **Start Small and Build Projects:** Begin with official tutorials and then create simple apps—like a to-do list or a weather dashboard—to apply what you learn.
2. **Understand the Core Concepts:** Focus on learning how components, state management, and reactivity work in each framework. This foundation will help you as projects get more complex.
3. **Experiment and Break Things:** Don't be afraid to tinker with code, try new features, or intentionally break things. Debugging mistakes is one of the best ways to deepen your understanding!

You've got this, Anthony!

In [41]:
```
#from openai import OpenAI
#client = OpenAI()

response = client.responses.create(
    model="gpt-4.1",
    tools=[{"type": "web_search_preview"}],
    input="What was a positive news story from today?"
)

print(response.output_text)
```

Today, October 13, 2026, a heartwarming story emerged from Montana, where a high school softball player, Samantha Rensvold, discovered that a member of the opposing team was her biological sister. Samantha, who was adopted and aware of her adoption since infancy, made this unexpected connection during a game, leading to a joyful reunion on the field. ([brightnewsdaily.com](https://brightnewsdaily.com/?utm_source=openai))

In other uplifting news, Barbie has honored tennis legend Serena Williams with a new signature doll. This initiative aims to inspire the next generation of women by celebrating Williams' achievements and contributions to sports and society. ([upliftingnewsapp.com](https://www.upliftingnewsapp.com/good-news-today/?utm_source=openai))

Additionally, a new app has been developed to help communities care for their local trees. Described as being "like caring for a pet," the app notifies users when neighborhood trees need watering, promoting environmental stewardship and community engagement. ([brightnewsdaily.com](https://brightnewsdaily.com/?utm_source=openai))

These stories highlight the positive events and innovations happening around the world today.

In [42]:
```
# My own prompt (retest) - web search
response = client.responses.create(
    model="gpt-4.1",
```

```
    tools=[{"type": "web_search_preview"}],
    input="Hi, I'm Anthony Bayate. What is one recent piece of news about Kean
University in New Jersey? Answer in 2-3 sentences."
)

print(response.output_text)
```

```
Kean University has been awarded a $500,000 National Science Foundation grant to
develop academic pathways and prepare students for the emerging CyberAI workforce.
This initiative will involve faculty from multiple disciplines to expand CyberAI
coursework, hands-on learning, and research opportunities. ([kean.edu]
(https://www.kean.edu/news?utm_source=openai))
```

In [43]:

```
import base64
from IPython.display import Audio, display
#from openai import OpenAI

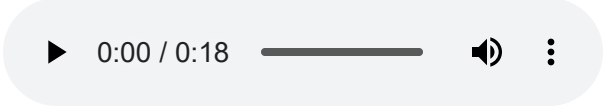
#client = OpenAI()

completion = client.chat.completions.create(
    model="gpt-audio-1.5",
    modalities=["text", "audio"],
    audio={"voice": "alloy", "format": "wav"},
    messages=[
        {
            "role": "user",
            "content": "Is a golden retriever a good family dog?"
        }
    ]
)

# printing choices[0] dumps thousands of base64 characters, so print the
transcript instead
print("Transcript:", completion.choices[0].message.audio.transcript)

wav_bytes = base64.b64decode(completion.choices[0].message.audio.data)
with open("dog.wav", "wb") as f:
    f.write(wav_bytes)
print("Saved dog.wav,", len(wav_bytes), "bytes")
display(Audio("dog.wav"))
```

```
Transcript: Golden Retrievers are often considered excellent family dogs. They're
usually friendly, patient, and good with children. Their gentle nature and love of
play can make them wonderful companions. Still, like any breed, it's important to
consider the time and energy needed for training, exercise, and care.
Saved dog.wav, 876044 bytes
```



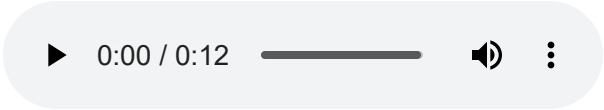
In [44]:

```
# My own prompt (retest) - audio
completion = client.chat.completions.create(
    model="gpt-audio-1.5",
    modalities=["text", "audio"],
    audio={"voice": "verse", "format": "wav"},
    messages=[{"role": "user", "content": "In two sentences, introduce Anthony
```

```
Bayate, a computer science student at Kean University, as if you were a podcast
host."}]
)

print("Transcript:", completion.choices[0].message.audio.transcript)
wav_bytes = base64.b64decode(completion.choices[0].message.audio.data)
with open("anthony_intro.wav", "wb") as f:
    f.write(wav_bytes)
print("Saved anthony_intro.wav,", len(wav_bytes), "bytes")
display(Audio("anthony_intro.wav"))
```

```
Transcript: Meet Anthony Bayate, a passionate computer science student at Kean
University. He's diving into algorithms and innovation, ready to share insights from
his journey through the world of code.
Saved anthony_intro.wav, 609644 bytes
```



check the files created (folder on the left)

Fix the error - find a working OpenAI model to describe the image and make it work

Error: `400 - Error while downloading file. Upstream status code: 403.` The model was fine; the URL `https://www.magnific.com/photos/cat-jpg` is a web page that refuses OpenAI's downloader, not a direct link to an image. **Fix:** use a direct `.jpg` link. To be safe I download the image myself and send it as a base64 data URL, so OpenAI never has to fetch it.

In [45]:
```
import base64, requests
from IPython.display import Image as ShowImage, display

image_url = "https://upload.wikimedia.org/wikipedia/commons/3/3a/Cat03.jpg"
img_bytes = requests.get(image_url, headers={"User-Agent": "Mozilla/5.0 (Colab
notebook)"}).content
display(ShowImage(data=img_bytes, width=300))
data_url = "" + base64.b64encode(img_bytes).decode()

response = client.responses.create(
    model="gpt-4.1-mini",
    input=[{
        "role": "user",
        "content": [
            {"type": "input_text", "text": "what's in this image?"},
            {"type": "input_image", "image_url": data_url},
        ],
    }],
)
```

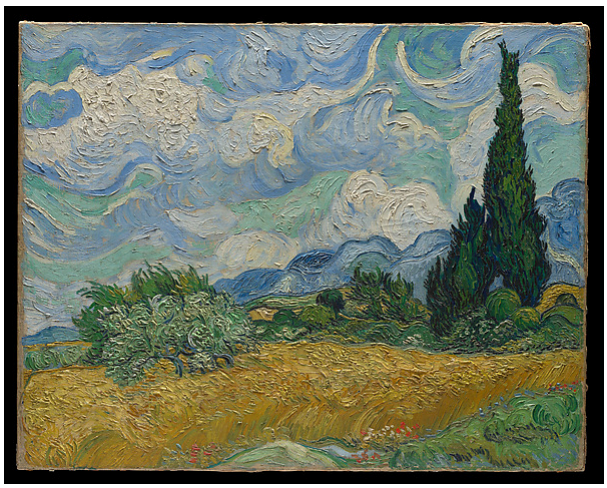
```
print(response.output_text)
```



```
This image shows a close-up of an orange tabby cat with light stripes. The cat has
yellow-green eyes and is looking directly at the camera. The background appears to
be an outdoor setting with a concrete-like surface and a blurry red hose in the
back.
```

In [46]:
```
# My own prompt (retest) - vision on a different image
image_url = "https://images.metmuseum.org/CRDImages/ep/web-large/DP-42549-001.jpg"
img_bytes = requests.get(image_url, headers={"User-Agent": "Mozilla/5.0 (Colab
notebook)"}).content
display(ShowImage(data=img_bytes, width=300))
data_url = "" + base64.b64encode(img_bytes).decode()

response = client.responses.create(
    model="gpt-4.1-mini",
    input=[{"role": "user", "content": [
        {"type": "input_text", "text": "I'm Anthony Bayate. Name this painting and
artist if you can, and describe it in 3 sentences."},
        {"type": "input_image", "image_url": data_url},
    ]}],
)
print(response.output_text)
```



```
This painting is "Wheatfield with Cypresses" by Vincent van Gogh. The artwork
depicts a vibrant wheat field with tall, dark cypress trees standing prominently to
the right. Above, swirling clouds and dynamic brushstrokes create a lively, almost
turbulent sky, reflecting van Gogh's intense emotional expression through vivid
color and texture.
```

Fix the error - find a working OpenAI model to create the image and make it work

Error: `400 - The model 'dall-e-3' does not exist.` DALL-E 3 has been retired. **Fix:** use `gpt-image-1-mini`. The GPT image models return the picture as base64 (`b64_json`) instead of a URL, so the code decodes it rather than downloading it. `img.show()` was removed because it opens an external viewer, which does nothing in Colab.

In [47]:
```python
#from openai import OpenAI
#client = OpenAI()
import base64
from io import BytesIO
from PIL import Image
from IPython.display import display

result = client.images.generate(
    model="gpt-image-1-mini",
    prompt="a white siamese cat",
    size="1024x1024",
    quality="low"
)

# GPT image models return base64, not a URL
img = Image.open(BytesIO(base64.b64decode(result.data[0].b64_json)))
img.save("siamese_cat.png")
display(img.resize((512, 512)))
```



In [48]:
```
# My own prompt (retest) - image generation
result = client.images.generate(
    model="gpt-image-1-mini",
    prompt="A friendly cartoon college student coding a web app on a laptop in a
cozy campus library, "
           "with a banner on the wall that says 'Anthony Bayate', flat
illustration style",
    size="1024x1024",
    quality="low"
)
img = Image.open(BytesIO(base64.b64decode(result.data[0].b64_json)))
img.save("anthony_coding.png")
display(img.resize((512, 512)))
```



Find any other OpenAI API feature and demonstrate its working

Feature: Embeddings (semantic search). An embedding turns text into a vector of numbers; texts with similar meaning get vectors that point the same way. Below I embed a few of my course topics and rank them by cosine similarity to a question, which is how RAG apps find relevant documents.

In [49]:
```
import math

docs = [
    "Vue uses reactive refs and computed properties to update the page.",
    "Svelte compiles components to plain JavaScript at build time.",
    "The NASA APOD API returns a new astronomy picture every day.",
    "Python functions can return multiple values as a tuple.",
    "A golden retriever is a friendly family dog.",
]
question = "Which JavaScript framework does its work at compile time?"

resp = client.embeddings.create(model="text-embedding-3-small", input=docs +
[question])
vectors = [d.embedding for d in resp.data]
```

```
q_vec, doc_vecs = vectors[-1], vectors[:-1]

def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    return dot / (math.sqrt(sum(x * x for x in a)) * math.sqrt(sum(y * y for y in
b)))

print(f"Anthony Bayate - embeddings demo ({len(q_vec)} dimensions per vector)")
print(f"Question: {question}\n")
for score, doc in sorted(((cosine(q_vec, v), d) for v, d in zip(doc_vecs, docs)),
reverse=True):
    print(f"{score:.3f}  {doc}")
```

```
Anthony Bayate - embeddings demo (1536 dimensions per vector)
Question: Which JavaScript framework does its work at compile time?

0.662  Svelte compiles components to plain JavaScript at build time.
0.274  Vue uses reactive refs and computed properties to update the page.
0.073  Python functions can return multiple values as a tuple.
0.059  The NASA APOD API returns a new astronomy picture every day.
0.016  A golden retriever is a friendly family dog.
```